Sun 14 Dec 2025

SpotMicro

Sources

- HALstm32g4.pdf > page=323
- refManual.pdf > page=91 → flash memory overview
- FLASH functional description

FLASH memory layout

Flash memory is organized in 2 blocks:

- Main memory block: 64 pages of size 2KB, starting from address `0x0800 0000`
- Information block, that comprises:
  - *System memory*: write protected, containst the bootloader used to re-program flash memory. 28KB, starting from `0x1FFF 0000`
  - *OTP area*: 1KB of memory for user data, programmable only once
  - *Option bytes*: 48 bytes for user configuration (`0x1FFF 7800 - 0x1FFF 782F`)

Where to flash firmware?

Since the bootloader also lives in FLASH memory, and it is required to be the first block of code, we must allocate some pages for it.
For now, I defined the size of the bootloader to be 16KBs, so program memory starts at `0x0800 0400`

Default value

All bits in FLASH memory are set to 1 by default!
So, empty flash memory holds 1s

Writing firmware to flash

> Firmware is flashed to FLASH memory through the FLASH library, provided by the HAL. The main features are:
>
> - Flash memory read, program and erase
> - Read and write protections
> - Option bytes programming
> - Prefetch on I-Code
> - Cache lines?

Firmware Transfer Rule

Formware is transmitted as a contiguous byte stream in ascending address order.
The first byte transmitted maps to the lowest Flash address of the application region

HAL_FLASH_Program

Definition

Here is the source code signature for the function `HAL_FLASH_Program`:

```
/**
* @brief Program double word or fast program of a row at a specified address with interrupt enabled.
```

```
 * @param TypeProgram Indicate the way to program at a specified address.
 * This parameter can be a value of @ref FLASH_Type_Program.
 * @param Address specifies the address to be programmed.
 * @param Data specifies the data to be programmed.
 * This parameter is the data for the double word program and the address where
 * are stored the data for the row fast program.
 *
 * @retval HAL_Status
 */
HAL_StatusTypeDef HAL_FLASH_Program(uint32_t TypeProgram, uint32_t Address, uint64_t Data)
```

What parameter `TypeProgram` should we provide?
In the sourcecode, we can find the following macro being asserted:

```
#define IS_FLASH_TYPEPROGRAM(VALUE) (((VALUE) == FLASH_TYPEPROGRAM_DOUBLEWORD) || \
((VALUE) == FLASH_TYPEPROGRAM_FAST) || \
((VALUE) == FLASH_TYPEPROGRAM_FAST_AND_LAST))
```

Where the defines are commented as follows:

- **Doubleworld**: Program a double-word (64-bit) at a specified address.
- **Fast and Last**: Fast program a 32 row double-word (64-bit) at a specified address.
- **Fast**: Fast program a 32 row double-word (64-bit) at a specified address. And this is the last 32 row double-word (64-bit) programmed

Writing Option Bytes

> Definition
>
> The option bytes are configured by the end user depending on the application requirements
> → refManual.pdf > page=200

They are not a tool for runtime state

Flags and bootloader state

The last page of bootloader code is reserved to data.
The first word of that page is the `bootloader_flag`, set by application to require bootloader mode and cleared by the bootloader itself.

Reading from FLASH

> Memory mapping
>
> FLASH is mapped to memory in these MCUs, so it's just like reading from RAM!